

Ruc

静的型付けプログラミング言語

えくれあ KADOKAWAドワンゴ情報工科学院高等部 2 年

動作環境 Runtime Environment

- x86_64 GNU/Linux PC
- glibc環境 (muslは不可)
- Debian系で動作確認済

依存関係：

- Rust, GCC, NASM
- pkg-config, GTK+3

```
ktop/ruc$ fastfetch
yyssso/:.
yyyyyyyyyyo:
yyyyyyyyyyso-
yoyyydmmMyyyyy-
MMMMMMMMMMMyyyyyyyo
poodMMMMMyyyyyyyo
yyyyyydMMYssssyyoy
yyyyyydMMMMMyyyy-
yyyyyyyyyydMMMMMyyyy
yyyyyyyyyyssosyyyyyy
yyyyyyyyyyssosyyyyyy
yyyyyyyyyydMMMMMyyyy
yyyyyydMMYoyoyoyoy
poodMMMMMyoyyyyyyyo
MMMMMMMMMMMyyyyyyyo
ysyydMMMyssyyss-
yyyyyyssosyyoy-
yyyyyyyyyyo/.
yyssso/:.`

x7ea@Computer
-----
OS: Kubuntu 26.04 LTS (Resolute Raccoon) x86_64
Host: PC-VJ25TLN21NRF
Kernel: Linux 7.0.0-34-generic
Uptime: 1 day, 6 hours, 15 mins
Packages: 3016 (dpkg), 10 (snap)
Shell: bash 5.3.9
Display (LGD033B): 1366x768 in 16", 60 Hz [Built-in]
DE: KDE Plasma 6.6.4
WM: KWin (Wayland)
WM Theme: Breeze
Theme: Breeze (Dark) [Qt], Breeze-Dark [GTK2], Breeze [GTK3/4]
Icons: breeze-dark [Qt], breeze-dark [GTK2/3/4]
Font: Noto Sans (10pt) [Qt], Noto Sans (10pt) [GTK2/3/4]
Cursor: breeze (30px)
Terminal: konsole 25.12.3
Terminal Font: Hack (12pt)
CPU: Intel(R) Core(TM) i5-3210M (4) @ 3.10 GHz
GPU: Intel 3rd Gen Core processor Graphics Controller @ 1.10 GHz
Memory: 6.62 GiB / 7.64 GiB (87%)
Swap: 511.75 MiB / 512.00 MiB (100%)
Disk (/): 38.35 GiB / 225.91 GiB (17%) - ext4
Local IP (wlp2s0): 192.168.200.39/24
Battery (PC-VP-WP114): 100% [AC Connected]
Locale: ja_JP.UTF-8
```



構文解析器 Parser & Lexer

- 再帰下降アルゴリズムで抽象構文木を構築
- 関数・構造体の定義、条件分岐、繰り返し
- RustやOCamlに似た文法（拡張子`.rc`）

app/reduce.rc

```
1 use std
2
3 fn reduce<T>(this: [T], lambda: T(T, T)): T {
4   let acc: T
5   for i = this do {
6     let acc = lambda(acc, i)
7   }
8   acc
9 }
10
11 fn add<T>(a: T, b: T): T a + b
12
13 fn main() {
14   let x = [8, 5, 4, 6, 2]
15   print reduce<Int>(x, add<Int>)
16
17   let y = [1.2, 2.3, 4.0]
18   print reduce<Float>(y, add<Float>)
19 }
```

app/prime.rc

```
1 use std
2
3 fn is_prime(n: Int): Bool {
4   if n < 2 then return false
5   if n < 4 then return true
6
7   let i = 2
8   while i < n do {
9     if n % i == 0 then return false
10    let i = i + 1
11  }
12  true
13 }
14
15 fn main() {
16   let n = input().atoi()
17   print is_prime(n)
18 }
```

```
while idx < chars.len() {
  let c = chars[idx];
  if esc {
    current.push(c);
    esc = false;
    idx += 1;
    continue;
  }
  if let Some(op) = src.get(idx..idx + 3)
    && ["< ", "> ", "≤ ", "≥ "].contains(&op)
  {
    if del == SPACE && level == 0 {
      tokens.append(&mut vec![current.clone(), op.trim().to_string()]);
      current.clear();
    } else {
      current += op;
    }
    idx += 3;
    continue;
  }
  match c {
    '<' | '(' | '{' | '[' if !quote => {
      if c.to_string() == del && level == 0 {
        tokens.push(current.clone());
        current.clear();
      }
      current.push(c);
      level += 1;
    }
    '>' | ')' | '}' | ']' if !quote => {
      current.push(c);
      level -= 1;
    }
    ''' => {
      quote = !quote;
    }
  }
}
```

型推論システム Type Inference

- 整数、浮動小数点数、真偽値、文字列、配列、構造体、列挙型
- 文脈で辞書に式と型情報に対応させ結果が一意に収斂する
- 多相型（ジェネリクス）は単相化エンジンを実装

```
fn main() {  
    let x = [8, 5, 4, 6, 2]  
    print reduce<Int>(x, add<Int>)  
  
    let y = [1.2, 2.3, 4.0]  
    print reduce<Float>(y, add<Int>)  
}  
x7ea@Computer:~/Desktop/ruc$ ./ruc.sh app/reduce.rc  
  
thread 'main' (69567) panicked at src/main.rs:34:18:  
Error! argument: Float(Float, Float) != Int(Int, Int)  
note: run with `RUST_BACKTRACE=1` environment variable to display  
backtrace  
/usr/bin/x86_64-linux-gnu-ld.bfd: /usr/lib/gcc/x86_64-linux-gnu/10/collect2.o: undefined reference to `main'  
collect2: error: ld returned 1 exit status  
./ruc.sh: 行 9: ./main: そのようなファイルやディレクトリはありません
```

```
Type::Class(Generic(name, args)) => {  
    let Some((params, table)) = ctx.global.table.get(&name) else {  
        return Err(format!("undefined: {name}"));  
    };  
    let (Object::Enum(mut layout) | Object::Struct(mut layout)) = table.clone();  
    for (_, field) in layout.iter_mut() {  
        for (arg, param) in args.iter().zip(params) {  
            *field = field.rewrite(param, arg);  
        }  
    }  
    let unify = match table {  
        Object::Enum(_) => Object::Enum(layout),  
        Object::Struct(_) => Object::Struct(layout),  
    };  
    ctx.global.table.insert(mangle.clone(), (Vec::new(), unify));  
}
```

コード生成器 Code Generating

- アセンブリ (intel記法) 直接出力
- 独自バックエンドでLLVM非依存
- AMD System V ABI 完全準拠
- C言語の関数を呼び出し可能

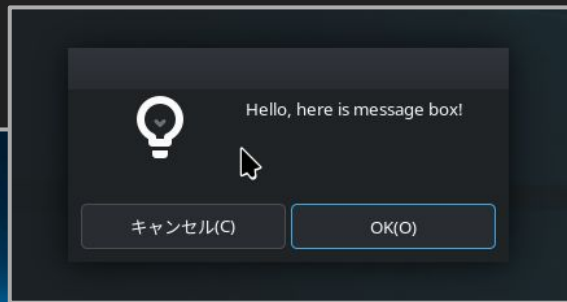
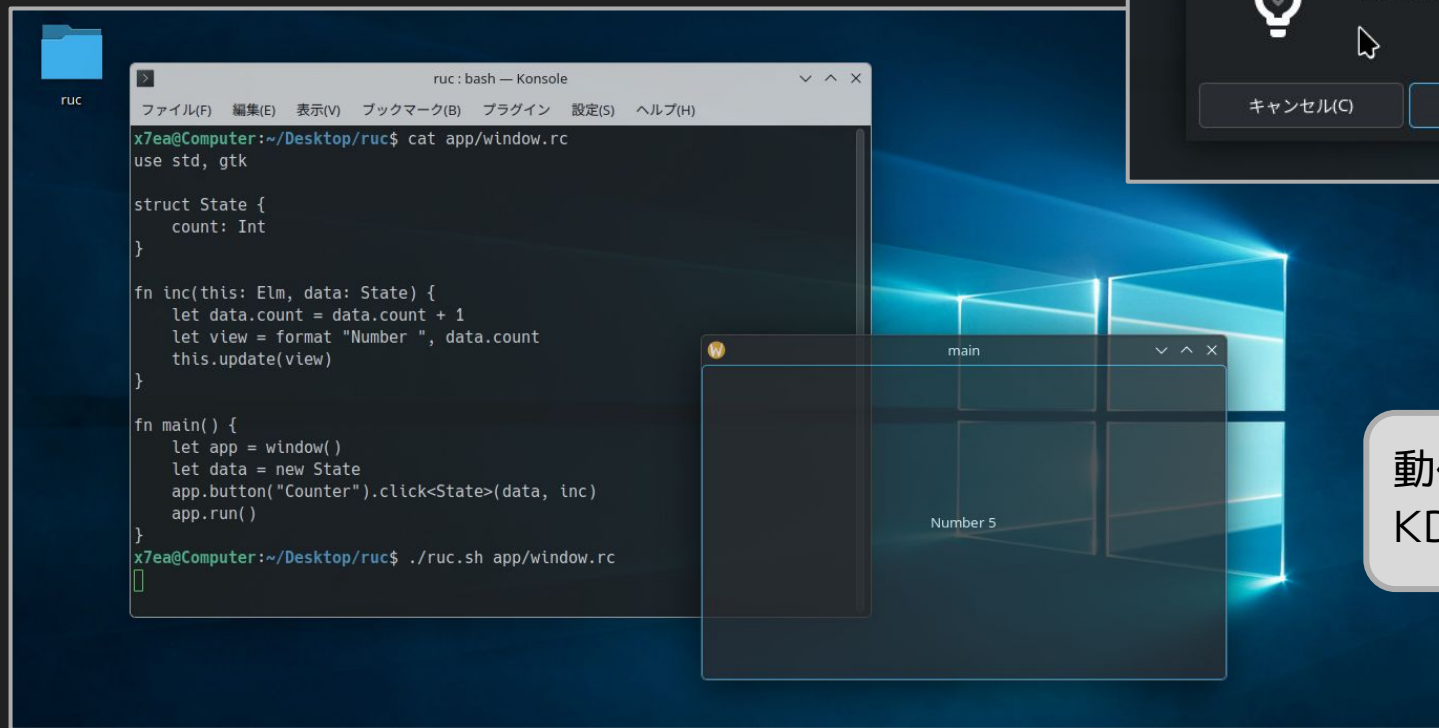
XOR命令で
最適化も実装

```
Expr::Integer(0) ⇒ Ok(String::from("\txor rax, rax\n")),
Expr::Integer(val) ⇒ Ok(format!("\tmov rax, {val}\n")),
Expr::Float(Float(0.0)) ⇒ Ok(String::from("\tpxor xmm0, xmm0\n")),
Expr::Float(val) ⇒ {
    let name = format!("float{}", label!());
    ctx.global.data += &format!("\t{name} dq {val:?}\n");
    Ok(format!("\tmovsd xmm0, [{name}]\n"))
}
Expr::String(val) ⇒ {
    let val = format!("\t{val}\n, 0")...
    let name = format!("str{}", label!());
    ctx.global.data += &format!("\t{name} db {val}\n");
    Ok(format!("\tmov rax, {name}\n"))
}
```

```
let (mut ptr, mut alloc) = (8, String::new());
let (mut idx, mut xmm) = (0, 0);
for (arg, (_, typ)) in args.iter().enumerate() {
    let arg = (arg as isize - 4) * 8;
    if typ == &Type::Float {
        if xmm < 8 {
            alloc += &format!("\tmovsd [rbp-{}ptr]], xmm{xmm}\n")
        } else {
            alloc += &format!("\tmovsd xmm0, [rbp+{}arg]]\n\tmovsd [rbp-{}ptr]], xmm0\n")
        };
        xmm += 1;
    } else {
        if let Some(reg) = ABI.get(idx) {
            alloc += &format!("\tmov [rbp-{}ptr]], {reg}\n")
        } else {
            alloc += &format!("\tmov rax, [rbp+{}arg]]\n\tmov [rbp-{}ptr]], rax\n")
        };
        idx += 1;
    }
    ptr += 8;
}
let body = body.emit(ctx)?;
ctx.table.insert(name.clone(), ctx.local.clone());

let size = ctx.local.var.len() * 8;
let frame = format!(
    "\tpush rbp\n\tmov rbp, rsp\n\tsub rsp, {}{}\n",
    if size % 16 == 0 { size } else { size + 8 }
);
Ok(format!("\t{name}:\n\t{frame}{alloc}{body}\tleave\n\tret\n\n"))
```

GTK+3と結合したGUIアプリ例



動作環境
KDE Plasma

ご清聴ありがとうございました！

GitHub: <https://github.com/x7ea/ruc>

DeepWiki: <https://deepwiki.com/x7ea/ruc>

今後の目標：

- メモリ管理（RAII）と所有権
- 標準ライブラリの拡充



Star & Watch
お願いします！